The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software

Student Name: AYUSHI SHUKLA

Registration Number: 24BCG10006

Slot: F11

<

INTRODUCTION

The foundation of modern software engineering relies heavily on tools that are not owned by any single corporation but are maintained by a global community of developers. This collaborative paradigm is the heartbeat of open source.

For this capstone project, I selected **Git** as the subject of my open source audit. This software was chosen because it fundamentally revolutionized how source code is managed, possessing a dramatic origin story born out of necessity, and serving as the absolute backbone of modern collaborative development.

This report documents my structured audit of Git across multiple dimensions: its creation and the critical problem it solved, its licensing model, the ethical dimensions of its ecosystem, its footprint within a Linux environment, its broader impact on the FOSS landscape, and a critical comparison against a proprietary version control alternative. Additionally, I have developed five shell scripts that demonstrate practical Linux administrative skills, reflecting the automation-first mindset of the open-source community.

PART A: origin and philosopy

A1. The problem this software was created to solve

History: Git was created by Linus Torvalds, the original developer of the Linux kernel. Development began in April 2005. It was built over a remarkably short period of just a few weeks to address an immediate crisis in the Linux kernel community.

The Problem: Prior to 2005, the Linux kernel development was managed using a proprietary distributed version control system called BitKeeper. The company behind BitKeeper revoked the free licenses granted to Linux developers after a dispute over reverse-engineering. Torvalds needed a replacement immediately but found that none of the existing free systems (like CVS or SVN) met his strict requirements. He needed a tool that was:

1. **Fully distributed (every developer has full local history).**
2. **Extremely fast (capable of handling the massive Linux kernel codebase).**
3. **Secure against accidental or malicious corruption (cryptographic integrity).**
4. **Capable of supporting non-linear development (thousands of parallel branches).**

Why Open Source: Torvalds released Git under an open-source license because:

1. **The tool was explicitly built to manage the Linux kernel, the world's most prominent open-source project.**
2. **He required rapid contributions from other kernel developers to make the tool viable as quickly as possible.**
3. **It aligned with his fundamental belief that infrastructure tools should be freely available to developers.**

<

The Philosophy: Git's design philosophy centers on performance, data integrity (using SHA-1 hashing), and treating branching and merging as fast, first-class operations rather than expensive, rare events.

Part A2: License Analysis

Git uses the GNU General Public License version 2.0 (GPLv2).

This means the software:

1. **Is a strong copyleft license.**
2. **Guarantees the freedom to run, study, share, and modify the software.**
3. **Requires that any distributed modifications or derivative works also be licensed under GPLv2.**
4. **Allows commercial use, provided the source code of distributed binaries remains open.**

History of License: Unlike projects that have changed licenses over time, Git has remained staunchly under GPLv2 since its inception. This matches the licensing of the Linux kernel itself, ensuring philosophical and legal alignment between the two projects Linus Torvalds created.

Practical Impact: The GPLv2 license ensured that while companies could build massive proprietary platforms *around* Git (like GitHub or Bitbucket), the core Git executable itself could never be enclosed or privatized. Any improvements made to the core Git engine by these massive corporations must be shared back with the community.

<

PART A 3: Ethical Discussion

PART A3: Ethical Discussion

Corporate Dominance: While Git is FOSS, the platforms that host it are heavily centralized and corporatized (e.g., Microsoft owning GitHub). This creates an ethical tension where open-source labor is heavily dependent on proprietary corporate infrastructure. **Maintainer Burden:** Despite being used by nearly every software company on Earth, the core maintenance of Git is handled by a very small team (led by Junio C Hamano), raising questions about corporate free-riding and the sustainability of critical FOSS infrastructure. **Ethical Dilemmas:** Because Git is decentralized and neutral, it is frequently used to host and distribute malware source code, stolen data, and controversial exploits. The community faces ongoing debates about censorship, platform de-platforming, and whether version control tools should have moral guardrails.

Part B - Linux Footprint

To understand how Git lives on a Linux system, I analyzed its installation on Ubuntu 22.04 LTS. The following sections document its installation method, key directories, and structure.

<

B1. Installation Method Ubuntu is based on Debian, so it utilizes the apt package manager. Git is usually available in the core repositories but is not always pre-installed on minimal server images. I installed it using sudo apt install git. I verified the installation with git --version, which returned git version 2.34.1.

B2. Key Directories Ubuntu follows the Filesystem Hierarchy Standard (FHS).

/usr/bin/git

- The main executable binary.

/usr/lib/git-core/

- Contains the actual core Git commands (e.g., git-commit, git-merge). Git uses a dashed form internally to manage its various sub-commands.

/etc/gitconfig

- The system-wide configuration file for all users.

~/.gitconfig

- The user-specific configuration file (where names and emails are stored).

/usr/share/doc/git/

- Standard location for local documentation and man pages.

<

Part C - FOSS Ecosystem

Git has fundamentally reshaped the open-source ecosystem.

Dependencies: Git relies on several other critical open-source libraries to function:

- zlib (for data compression).
- OpenSSL or GnuTLS (for secure HTTPS connections).
- curl (for network transfers).

What Git Enabled:

- **Social Coding:** It paved the way for platforms like GitLab, Gitea, and GitHub, which gamified and socialized open-source contributions.
- **Modern CI/CD:** Continuous Integration and Deployment pipelines are entirely triggered by Git hooks and commit states.
- **Infrastructure as Code (IaC):** Treating server configurations as versioned software via GitOps.

Community and Governance: Git is a member project of the **Software Freedom Conservancy (SFC)**, which provides legal and financial backing. The technical governance is meritocratic, led by the core maintainer Junio C Hamano, with decisions made via a public mailing list.

Open Source vs Proprietary

<

Open source (python) :

Cost : free

Learning curve : gentel design for readablity

Ecosystem : 400000 + package on pypi

Freedom : full source access

Support : community + commercial

Security : anyone can audit code

Community vs coorperate : community elected

Proprietary (MATLAB):

Cost : 2100 dolars per year

Learning curve : steep for programing concepts

Ecosystem : limited to mathworks toolboxes

Freedom : no source access

Support : paid support only

Security : only vendor’s internal team can audit the code

Community vs coorporate control :single coorporation eg mathworks

<

Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

```
#1/bin/bash

echo "System Info"

echo "User: $(whoami)"

echo "kernel: $(uname -r)"

echo "Date: $(date)"

echo "Uptime: $(uptime -p)"
```

```
User: bhumija
Kernel: 6.12.38+kali-amd64
Date: Monday 30 March 2026 09:09:02 PM IST
Uptime: up 21 minutes
```

This script serves as a welcome screen which displays essential information about the Linux system .it introduces the user to the system showing the Linux distribution name kernel version ,home directory system uptime etc .in terms of shell scripting this script demonstrates the use variables to store the command outputs and also showcases how to extract information from system files .this script establishes the foundation for understand how shell script interact with the system .

<

Script 2 — FOSS Package Inspector

```
#2/bin/bash

PACKAGE = "python3"

if dpkg -l| grep -q $PACKAGE
then
   echo "$package is installed"
   python3 --version
else
   echo "$PACKAGE isNOT installed"
Fi
```

<

<

```
└─$ ./script2.sh
python3 is installed
Python 3.13.12
```

This scripts checks weather the python is installed in the system and displays detailed information about it . this script begins with the package pyton3 which stores the name which needs to be checked and the uses the the if else conditional statement to check weather the package is installed .

<

Script 3 — Disk and Permission Audit

```
#!/bin/bash

echo "----- Disk Audit -----"
echo ""

for dir in /etc /home /usr/bin
do
   echo "Checking: $dir"
   echo "Disk Usage:"
   du -sh "$dir" 2>/dev/null
   echo "Permissions & Owner:"
   ls -ld "$dir"
   echo "----------------------------"
   echo ""
done
```

<

```
# Additional check for Python config directory
echo "----- Python Configuration Audit -----"
echo ""
PYTHON_CONFIG="/etc/python3"
if [ -d "$PYTHON_CONFIG" ]; then
   echo "Checking: $PYTHON_CONFIG"
   du -sh "$PYTHON_CONFIG" 2>/dev/null
   ls -ld "$PYTHON_CONFIG"
   echo ""
   echo "Contents (first 10 items):"
   ls -la "$PYTHON_CONFIG" | head -10
else
   echo "$PYTHON_CONFIG does not exist"
fi
```

```
──── Disk Audit ────

hecking: /etc
isk Usage:
5M      /etc
ermissions & Owner:
rwxr-xr-x 181 root root 12288 Mar 30 20:48 /etc
──────────────────────────────

hecking: /home
isk Usage:
19M     /home
ermissions & Owner:
rwxr-xr-x 3 root root 4096 Sep  1  2025 /home
──────────────────────────────

hecking: /usr/bin
isk Usage:
04M     /usr/bin
ermissions & Owner:
rwxr-xr-x 2 root root 135168 Mar 29 01:37 /usr/bin
──────────────────────────────

──── Python Configuration Audit ────

hecking: /etc/python3
.0K     /etc/python3
rwxr-xr-x 2 root root 4096 Sep  1  2025 /etc/python3

ontents (first 10 items):
otal 20
rwxr-xr-x   2 root root  4096 Sep  1  2025 .
rwxr-xr-x 181 root root 12288 Mar 30 20:48 ..
rw-r--r--   1 root root    94 Sep  1  2025 debian_config
```

<

The script preform basic disk audit by performing looping 3 important system directories . this script provides aquick overview of how much space the critical system directories are consuming and how owns them which is valuable for security

<

Script 4

```
#!/bin/bash

# Default values for log file and keyword
LOGFILE=${1:-"/var/log/messages"}
KEYWORD=${2:-"error"}
LINES=${3:-5}  # Default to 5 lines of matching output
DATE_FILTER=${4}  # Optional: filter by date

# Print header
echo "========================================"
echo "        LOG FILE ANALYZER"
echo "========================================"
echo "File: $LOGFILE"
echo "Keyword: $KEYWORD"
echo "Matching Lines to Show: $LINES"
echo ""

# Check if the log file exists and is readable
if [ ! -f "$LOGFILE" ]; then
```

<

```
    echo "Error: File '$LOGFILE' not found or not readable."
    exit 1
fi

echo "Log file found."

# If a date filter is provided, apply it.
if [ -n "$DATE_FILTER" ]; then
    echo "Filtering logs by date: $DATE_FILTER"
    MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" | grep
"$DATE_FILTER")
else
    MATCHES=$(grep -i "$KEYWORD" "$LOGFILE")
fi

# Count the number of matches
COUNT=$(echo "$MATCHES" | grep -c .)

echo "Keyword '$KEYWORD' found $COUNT times."
```

<

```
~$ ./script4.sh sample.log error
═══════════════════════════════════
        LOG FILE ANALYZER
═══════════════════════════════════
File: sample.log
Keyword: error
Matching Lines to Show: 5

Log file found.
Keyword 'error' found 2 times.

Last 5 Matching Lines:
ERROR: Failed to connect to database
ERROR: Timeout occurred
```

<

The script analyzes the log file by reading it line by line , counting how many lines contain error and displaying the total count . this script is the practical tool for system adminstrator who need to quickly access the error frequency .

<

Script 5

```
#!/bin/bash

read -p "Enter Tool name:" tool
read -p "What does freedom mean to you?" freedom
read -p "What do you want to build $build."> build

echo "I use $tool daily. Freedom means $freedom.I
want to build $build."> manifesto.txt

echo"Manifesto saved in mainfesto.txt"
cat manifesto.txt
```

<

```
Enter Tool name:tool
What does freedom mean to you?freedom
What do you want to build .build
./script5.sh: line 9: echoManifesto saved in mainfesto.txt: command not found
I use tool daily. Freedom means freedom.I want to build .
```

This is an interactive script it generates a personalized open source philosopy satement by asking the user 3 questions and composing their answers into a formatted menifesto document .the script beging by displaying a welcome banner and then the name of the tool , what fr

<